

Работа с двигателями

В прошлом уроке мы смогли получить изображение с камеры и передать его на ПК. Теперь займемся тем, что будем управлять движением двигателей с помощью python-скрипта.

1. Управление L298N

L298N - это драйвер двигателя, который позволяет подключить к нему два двигателя, чтобы ими управлять через пины IN1, IN2, IN3, IN4, которые подключаются к выводам DC-двигателя через соответствующие клеммы.

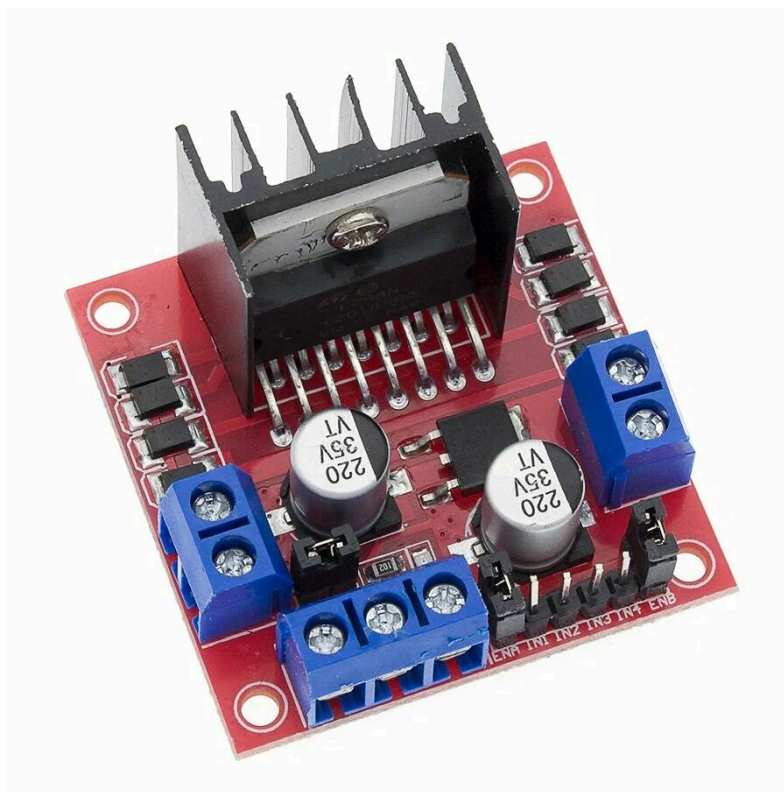


Рисунок 1 - Плата L298N

Но зачем нам использовать эту плату, если можно подключиться к двигателю напрямую? Есть два момента:

- 1) Мощность (ток и напряжение), выдаваемое ESP32 недостаточна, чтобы запустить хотя бы один двигатель, а L298N позволяет подключить внешнее питание, которое и будет управлять двигателем, мы его только перенаправлять;

2) Т.к. двигатель в процессе работы генерирует обратное ЭДС (напряжение), которое может привести к тому, что либо сгорит ножка микроконтроллера, либо вся плата целиком.

По этим причинам мы используем драйвер двигателя.

Проверьте подключение! Вот пример, как должно быть:

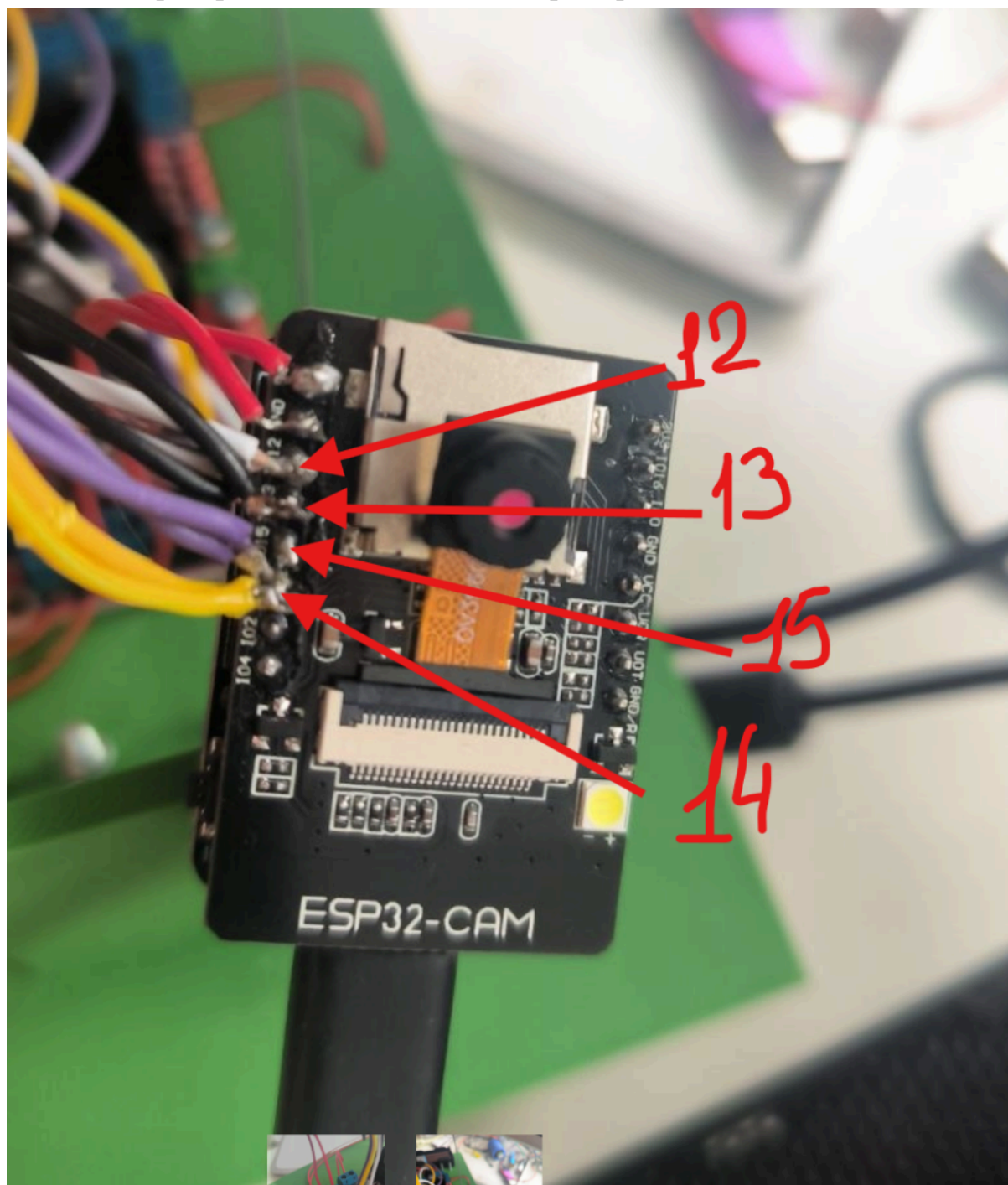


Рисунок 2 - Подключение к ESP32

Пины и провода:

- 1) Белый - GPIO12;
- 2) Черный - GPIO13;
- 3) Фиолетовый - GPIO15;
- 4) Желтый - GPIO14.

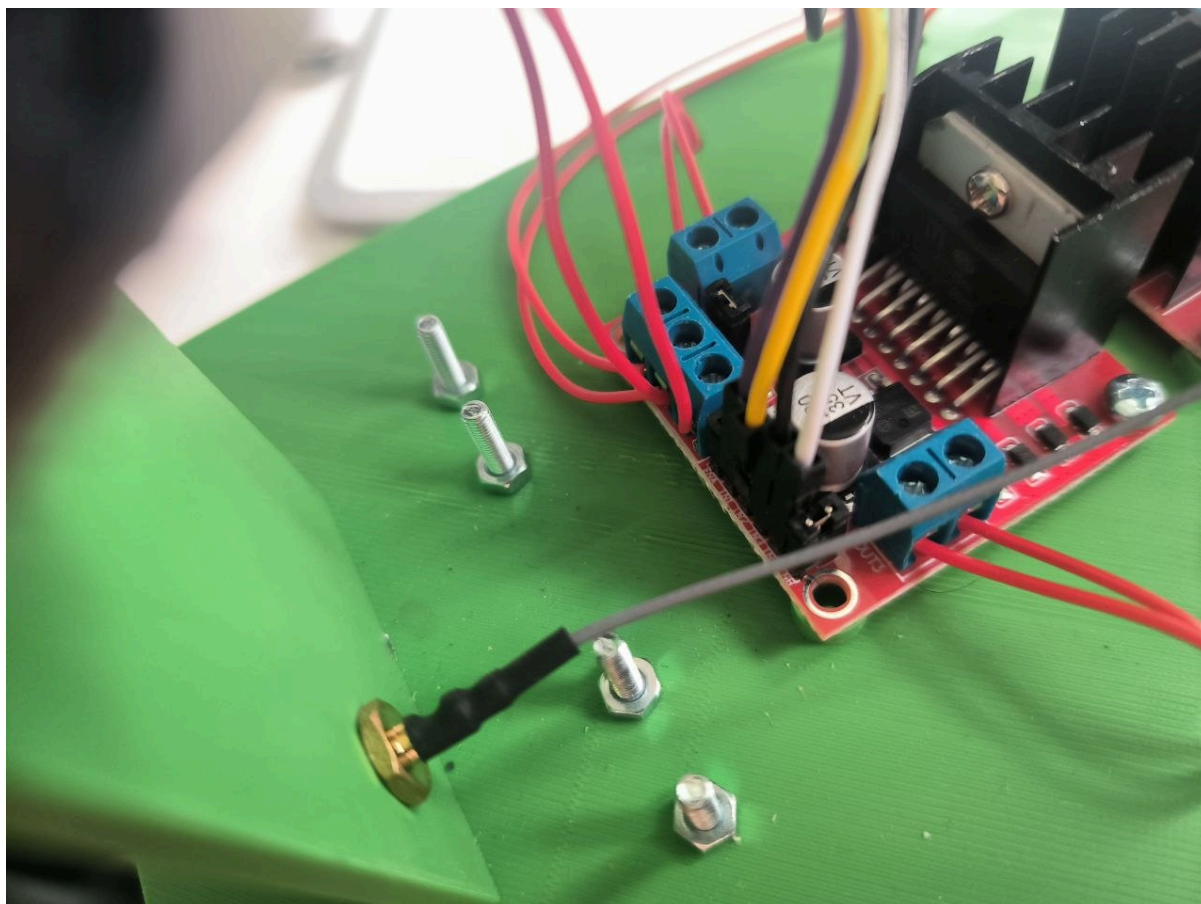


Рисунок 3 - Подключение к ближнему к EP32 L298N

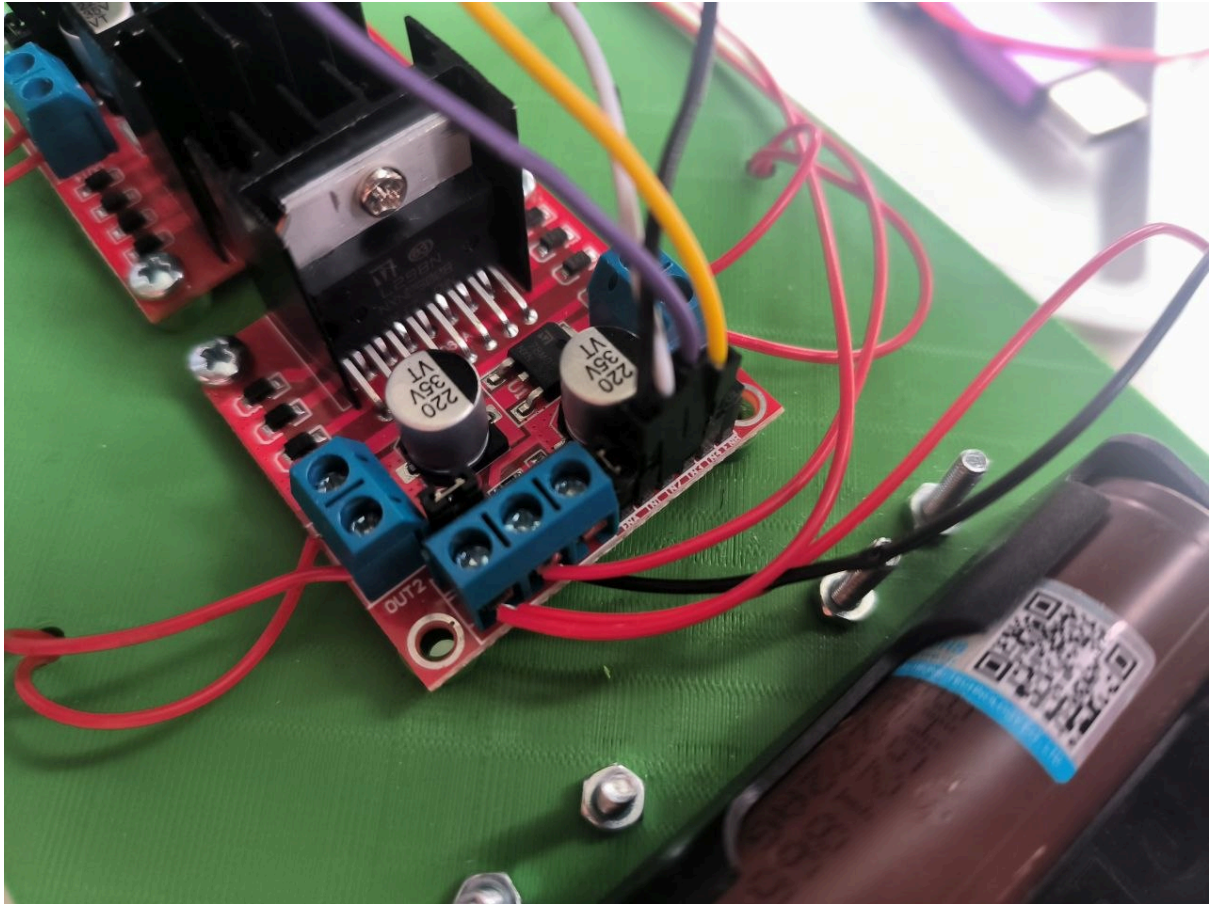


Рисунок 4 - Подключение к дальнему от EP32 L298N

Если что-то у вас не совпадает, то мы это проверим в процессе написания прошивки.

Открывайте Arduino IDE/VSCode - начнем писать прошивку. Первое, с чего начнем, напомним класс, который будет отвечать за управление двигателями. Создайте файл **DriverManager.hpp**. Запишем в него основу:

```
#ifndef DRIVERMANAGER_HPP_
#define DRIVERMANAGER_HPP_

class DriverManager {
private:
    // — Пины моторов
    #define RIGHT_IN1 12
    #define RIGHT_IN2 13
    #define LEFT_IN3 14
    #define LEFT_IN4 15
    #define STATUS_LED 33
```

```

public:
    // Конструктор класса DriverManager
    DriverManager()
    {
    }
};

```

Внутри класса определяем пины, которыми будем управлять.

Разберем, как мы будем управлять двигателями и напомним функцию управления.

	Мотор 1		Мотор 2	
	IN1	IN2	IN3	IN4
Вперёд	HIGH/PWM	LOW	HIGH/PWM	LOW
Назад	LOW/PWM	HIGH	LOW/PWM	HIGH
Холостой	LOW	LOW	LOW	LOW
Тормоз	HIGH	HIGH	HIGH	HIGH

Рисунок 5 - Таблица управления L298N

По таблице видно, что управление двигателями осуществляется переключением пар IN1/IN2 и IN3/IN4. Мы будем использовать три из четырех функций: вперед, назад и холостой ход. Попробуем перенести эту логику в код:

```

// Управление двигателями
void move(int leftDir, int rightDir)
{
    // 1) больше 0 - движение вперед
    // 2) меньше 0 - движение назад
    // 3) 0 - остановка

    // Движение вперед (ЛЕВЫЙ)
    // Пример: если leftDir > 0, то LEFT_IN3 HIGH, иначе LOW
    // Остальное по аналогии
    digitalWrite(LEFT_IN3, leftDir > 0 ? HIGH : LOW);
    // Движение назад (ЛЕВЫЙ)
    digitalWrite(LEFT_IN4, leftDir < 0 ? HIGH : LOW);
    // Движение вперед (Правый)
    digitalWrite(RIGHT_IN1, rightDir > 0 ? HIGH : LOW);
}

```

```

// Движение назад (Правый)
digitalWrite(RIGHT_IN2, rightDir < 0 ? HIGH : LOW);

// Остановка
if (leftDir == 0) {
    digitalWrite(LEFT_IN3, LOW);
    digitalWrite(LEFT_IN4, LOW);
}
if (rightDir == 0) {
    digitalWrite(RIGHT_IN1, LOW);
    digitalWrite(RIGHT_IN2, LOW);
}
}
}

```

В качестве аргументов мы передаем два значения - способ движения левого двигателя и способ движения правого двигателя. Если мы передаем значение больше 0 (**digitalWrite(LEFT_IN3, leftDir > 0 ? HIGH : LOW), digitalWrite(RIGHT_IN1, rightDir > 0 ? HIGH : LOW)**), то, согласно таблице, двигатели поедут вперед, меньше 0 - назад (**digitalWrite(LEFT_IN4, leftDir < 0 ? HIGH : LOW), digitalWrite(RIGHT_IN2, rightDir < 0 ? HIGH : LOW)**), 0 - стоп (**if (leftDir == 0) {**

digitalWrite(LEFT_IN3, LOW); digitalWrite(LEFT_IN4, LOW); **}).** Теперь разберем, что за странная конструкция: **rightDir < 0 ? HIGH : LOW**. Это тернарная операция - то же самое, что и “if-else”. Значение после “?” принимается, если утверждение **ВЕРНОЕ**, если утверждение **НЕ ВЕРНОЕ**, то применяется значение после “:”. Функция управления на этом завершена.

Добавим функцию инициализации пинов:

```

void init()
{
    pinMode(RIGHT_IN1, OUTPUT);
    pinMode(RIGHT_IN2, OUTPUT);
    pinMode(LEFT_IN3, OUTPUT);
    pinMode(LEFT_IN4, OUTPUT);
    move(0, 0);
}

```

Мы настраиваем пины, и на всякий случай указываем, что движения не должно происходить, путем вызова функции **move(0, 0)** с аргументами в

виде нулей. Класс управления завершен, теперь нам нужно все это интегрировать в основной код и добавить взаимодействие с сервером. Но сначала проверим, что все работает. Перейдите в основной цикл и вызовите функцию `move(1, 1)`. Нужно подключить `DriverManager.hpp` к основному файлу, создать объект `DriverManager driver;` и вызовите функцию `driver.move(1, 1)`. Если двигатели с обеих сторон крутятся “вперед”, то все верно. Если крутятся в противоположную сторону, то переставьте управляющие пины местами для каждого конкретного двигателя. После проверки удалите `driver.move(1, 1)`.

2. Модификация класса WifiManager

Т.к. мы планируем управлять роботом через сервер, развернутый на вашем ПК, то мы должны настроить прием и расшифровку команд от него. Для этого нам нужно модифицировать класс `WifiManager`.

Первое, что мы сделаем - добавим структуру, которая будет хранить состояние команды:

```
struct RobotCommands {  
    int leftMotor = 0; // -1 назад, 0 стоп, 1 вперед  
    int rightMotor = 0; // -1 назад, 0 стоп, 1 вперед  
    bool cameraEnabled = false;  
    int cameraInterval = 100; // мс между кадрами  
};
```

Как видно, команда будет хранить поведение левого/правого моторов, доступ к камере и интервал съемки камеры. Камеру пока по умолчанию отключен, т.к. пока она будет мешать нашему управлению, т.к. забирает много процессорного времени, но мы это исправим в следующий раз.

Создайте переменную внутри класса, которая будет хранить состояние команд:

```
RobotCommands commands; // Хранение состояния команд
```

Чтобы отключить камеру, перейдите в функцию `process()` и добавьте условие ее отключения:

```
// Доступна ли камера?  
if (commands.cameraEnabled) {
```

```

// Если да, то отправляем изображение
send(aFormat, aFb);
}

```

Теперь нужно написать парсер (функцию расшифровки) команды:

```

// Парсинг команд с сервера
void parseServerCommands(String response) {
    // Ожидаем формат: "LEFT:1,RIGHT:1,CAM:1,INTERVAL:500"
    int leftIdx = response.indexOf("LEFT:");
    int rightIdx = response.indexOf("RIGHT:");
    int camIdx = response.indexOf("CAM:");
    int intervalIdx = response.indexOf("INTERVAL:");

    // Обрабатываем данные, полученные от сервера и заполняем структуру "commands"
    if (leftIdx >= 0) {
        commands.leftMotor = response.substring(leftIdx + 5, response.indexOf(',', leftIdx)).toInt();
    }
    if (rightIdx >= 0) {
        int endIdx = response.indexOf(',', rightIdx);
        if (endIdx < 0) endIdx = response.length();
        commands.rightMotor = response.substring(rightIdx + 6, endIdx).toInt();
    }
    if (camIdx >= 0) {
        commands.cameraEnabled = (response.substring(camIdx + 4, response.indexOf(',',
camIdx)).toInt() == 1);
    }
    if (intervalIdx >= 0) {
        commands.cameraInterval = response.substring(intervalIdx + 9).toInt();
    }

    Serial.printf("Команды: L=%d, R=%d, CAM=%d, INT=%d\n",
        commands.leftMotor, commands.rightMotor,
        commands.cameraEnabled, commands.cameraInterval);
}

```

Объяснение:

int leftIdx = response.indexOf("LEFT:"); - мы берем строковую переменную запроса, ищем ее по слову "LEFT:" - рядом с этим словом лежит значение, которое отправил сервер.

```

if (leftIdx >= 0) {
    commands.leftMotor = response.substring(leftIdx + 5, response.indexOf(',', leftIdx)).toInt();
}

```

- **leftIdx** - это позиция, с которой начинается слово "LEFT:". Мы отступаем от начала на 5 знаков, чтобы получить число. С помощью **toInt()**

переводим из строки в integer (целочисленную) переменную. Все остальные поля команды парсятся аналогичным способом, но со своими особенностями.

Парсинг готов, теперь нужно добавить эту функцию для получения запроса, но для начала модифицируем **requestDataFromServer**:

```
// Прием всех данных с сервера
String requestDataFromServer(){
// .....
// Получаем ответ с сервера
String response = http.getString();

// Если ответ есть, то пишем об этом в консоль
if(httpCode > 0) {
    Serial.print("Данные получены! Код: ");
    Serial.println(httpCode);
    // При отсутствии ответа тоже
} else {
    Serial.print("Ошибка запроса: ");
    Serial.println(httpCode);
}

http.end();
// Возвращаем запрос
return response;
}
```

Теперь функция возвращает ответ сервера после своего выполнения, а не просто выводит его в консоль. В функции **void request()** добавим обработку запроса:

```
// Обрабатываем сообщения с сервера
void request()
{
    Serial.println("\nЗАПРАШИВАЮ ДАННЫЕ С СЕРВЕРА...");
    // Получаем ответ с сервера
    String response = requestDataFromServer();
    // Если ответ есть, то мы его обрабатываем
    if (response.length() > 0) {
        parseServerCommands(response);
        // и выставляем флаг об обновлении данных
        needUpdate = true;
    }
}
```

```
}  
}
```

Мы получаем запрос и, если он что-то внутри себя имеет, то мы его обрабатываем. У нас появилась переменная `needUpdate`, которая будет нам маркером того, что пришла новая команда и ее надо обработать. Добавьте булеву переменную в класс по аналогии с **commands**.

Почти все, осталось добавить функции, чтобы мы могли получать статус обновления команды и саму команду:

```
// Получить обработанную команду с сервера
```

```
RobotCommands getCommand()  
{  
    needUpdate = false;  
    return commands;  
}
```

```
// Проверка на наличие новых данных
```

```
bool isNeedUpdate()  
{  
    return needUpdate;  
}
```

Если мы получаем команду с помощью `getCommand()`, то мы считаем, что команда обработалась и меняем флаг на FALSE.

3. Основной алгоритм

Переходим в основной файл, подключаем файл управления двигателями и создаем объект класса:

```
// Объявление переменных управление WiFi, камерой и двигателем
```

```
WifiManager wifi;  
Camera camera;  
DriverManager driver;
```

В функции **setup** добавьте инициализацию двигателей:

```
// Инициализация двигателей (пинов, отвечающих за их управление)  
driver.init();
```

Весь код, который у вас сейчас в `loop()`, перенесите в функцию `void cameraHandler()`. Потом вызовите эту функции в `loop()`. Этим действием мы четко разделяем код управления камерой и двигателями.

То же самое делаем и для управления двигателями:

```
void driverHandler()
{
    // Проверяем, пришла ли новая команда с сервера на управление двигателем
    if (wifi.isNeedUpdate()) {
        // Получаем команду
        WifiManager::RobotCommands commands = wifi.getCommand();
        // Управляем моторами
        driver.move(commands.leftMotor, commands.rightMotor);
    }
}
```

Мы проверяем, пришла ли новая команда. Если она пришла, то мы записываем в переменную `commands` текущее состояние команды и передаем его в функции управления двигателями.

Итоговый `loop()`:

```
void loop() {

    // Обработчики управлением камерой и двигателями
    cameraHandler();
    driverHandler();
}
```

На этом все, в следующий раз займемся тем, что будем оптимизировать код, чтобы съемка камеры и обработка запросов работало вместе как можно стабильнее.